



Formal Verification

Final Report



MEZZANINE

Mezzanine Protocol

May 2026

Prepared for Mezzanine

Table of Contents

Project Summary.....	5
Project Scope.....	5
Project Overview.....	5
Protocol Overview.....	6
Threat and Security Overview.....	7
Solvency and Accounting Consistency.....	7
Oracle Usage and Correctness.....	8
ERC4626 Correctness, Reward Distribution and Loss Socialization.....	9
Governance Actions and Access Control.....	11
Epoch Advancement and Automation.....	12
Withdrawals and Claims.....	13
Findings Summary.....	16
Severity Matrix.....	16
Detailed Findings.....	17
Medium Severity Issues.....	20
M-01: applyLoss is going to double count the uncoveredMZ within totalUncoveredLossMZ.....	20
M-02: Some tranches may be able to escape the loss using instant redemptions.....	21
M-03: Tranche deposit limit bypass due to using live totalSupply can be used to manipulate yield distribution.....	22
M-04: Same report usage for two different epochs can result in double counting loss/yield.....	24
M-05: Insufficient validation of oracles synchronization.....	26
Low Severity Issues.....	28
L-01: Missing implementation to handle the totalUncoveredLossMZ.....	28
L-02: maxWithdraw and maxRedeem do not comply with eip4626.....	29
L-03: Tranche virtual shares may accrue excessive yield, which may eventually be stuck.....	30
L-04: Tranche inflation attack via truncation.....	31
L-05: setOracles() could be used under timelock to increase predictability.....	32
L-06: Retroactive admin withdraw fee change could disturb claims.....	33
L-07: checkUpkeep will return an incorrect output in multiple cases.....	34
L-08: Oracle design shall be improved in order to preserve solvency.....	35
L-09: All liquidity can be instantly withdrawn from the reservoir by allowed entities.....	36
L-10: applyLoss may not restore solvency in an edge case.....	37
Informational Severity Issues.....	38
I-01: Users can invoke claim for other tranches to increase their loss.....	38
I-02: Unnecessary _withdraw() validation.....	39
I-03: Allowlist contract does not call _disableInitializers().....	40
I-04: IssuanceVault fee configuration can be set incorrectly.....	41
I-05: withdrawnTo mapping may be inconsistent.....	42

I-06: setPolPayout shall not allow setting the payout address to address(this).....	43
I-07: Missing implementation to remove a tranche could cause DOS.....	45
I-08: Fee is rounded down, making it profitable to split withdrawals.....	47
Formal Verification.....	48
Verification Methodology.....	48
Verification Notations.....	49
General Assumptions.....	49
General Limitations.....	50
Formal Verification Properties Overview.....	51
Detailed Properties.....	57
Global Invariants.....	57
P-03. Solvency.....	57
P-05. No USDC drain.....	58
P-06: Integrity of distributeYield.....	58
P-07: Epoch ID non-decreasing.....	59
Tranche.....	59
P-22. Integrity of Tranche total assets.....	60
P-23. Integrity of Tranche requestWithdrawal.....	60
P-24. Exchange rate of Tranche shares only increases except for loss events.....	61
P-25. userSharesByEpoch tracked correctly.....	61
P-26. Deposit limit cannot be inflated.....	62
P-27. No double claiming.....	62
P-59. Tranche ERC4626 compatibility.....	63
P-60. Integrity of Tranche.finalizeEpoch.....	64
P-61. Unaccrued yield decreases.....	64
P-66. Anyone can call deposit, withdraw and claim.....	65
IssuanceVault.....	65
P-18. Sum of Tranche BPS values is equal to 10000 BPS.....	65
P-19. distributeYield() can be called at most once per epoch.....	66
P-20. Integrity of rebalance().....	66
P-21. Valid state conditions and helper invariants.....	67
LossManager.....	69
P-32. Integrity of applyLoss().....	69
P-36. No partial loss absorption.....	70
MZUSD.....	70
P-65. Integrity of mint and redeem.....	71
P-10. No MZUSD token loss.....	71
P-11. Rounding in MZUSD always favours the protocol.....	72
P-14. Integrity of deposit.....	73
P-15. USDC used in window is never greater than global redemption cap.....	73

P-16. windowStartTimestamp can only increase.....	74
P-30. Integrity of requestWithdrawal.....	74
NavOracleAutomation.....	74
P-38. Integrity of NavOracleAutomation.performUpkeep.....	75
P-39. performUpkeep() or rejectPendingNav() clear pendings fields.....	75
P-40. Integrity of submitNav().....	76
P-56. NAV value returned from oracle cannot be used in two epochs.....	76
P-57. Integrity of checkUpkeep().....	77
PauseManager.....	78
P-49. Idempotence of PauseManager.....	78
PolJuniorTranche.....	78
P-47. Integrity of notifyYield.....	79
P-69. Payout address must not be PolJuniorTranche contract.....	79
ReservoirUSDC.....	79
P-67. Integrity of ReservoirUSDC accounting.....	80
P-44. Integrity of ReservoirUSDC accounting.....	80
Disclaimer.....	82
About Certora.....	82

Project Summary

Project Scope

Project Name	Repository Link	Initial Commit Hash	Final Commit Hash	Platform
Mezzanine Protocol	https://github.com/woof-software/mezzanine-sc/	e675d7d769	c26be54426	Solidity

Project Overview

This document describes the specification and verification of **Mezzanine Protocol** using the Certora Prover. The work was undertaken from **March 15th** to **April 24th**.

The following contract list is included in our scope:

contracts/automation/NavOracleAutomation.sol
contracts/automation/UmbrellaEpochFinalizer.sol
contracts/pause/PauseCategory.sol
contracts/pause/PauseManagedUpgradeable.sol
contracts/pause/PauseManager.sol
contracts/Allowlist.sol
contracts/IssuanceVault.sol
contracts/LossManager.sol
contracts/MZUSD.sol
contracts/PolJuniorTranche.sol
contracts/ReservoirUSDC.sol
contracts/Tranche.sol

Protocol Overview

The Mezzanine Protocol receives USDC by allowed users and mints MZUSD (18 decimals) 1:1 with a $1e12$ scale, ensuring consistent value representation across differing decimal systems. Users can deposit their MZUSD tokens into tranches to earn rewards, where tranches follow the ERC-4626 model and support both direct and umbrella withdrawals. The tranche share tokens are 21 decimals, with a 3 decimal offset (`_decimalsOffset() = 3`), providing higher precision for share accounting relative to the underlying asset.

Two oracles are used to determine whether the NAV has increased or decreased: a NAV oracle and a total supply oracle. If rewards have been accrued (i.e., NAV exceeds supply), they are distributed among tranches based on their TVL and risk factor. If there is a loss (i.e., NAV falls below supply), it is applied to tranches according to their hierarchy, typically impacting junior tranches first before propagating upward.

A Reservoir stores the USDC liquidity, and whitelisted strategies can withdraw from it to deploy capital and generate yield. Users interact with the system through ERC-4626 tranches, choosing between direct or umbrella withdrawals depending on liquidity and aggregation preferences.

Threat and Security Overview

Solvency and Accounting Consistency

Affected Components: IssuanceVault.sol, LossManager.sol, MZUSD.sol, ReservoirUSDC.sol, Tranche.sol, NavOracleAutomation.sol.

Risk Description: For every MZUSD minted, there must be exactly 1 USDC held as backing collateral, ensuring full reserve parity at all times. All user-facing operations, including minting and withdrawals, must strictly preserve this invariant without exception.

Additionally, oracle reporting mechanisms must accurately and continuously track the Net Asset Value (NAV). This ensures that any yield generation is correctly reflected in token supply, and that potential losses are fairly and transparently distributed across the respective tranches.

Key Considerations:

- **Rounding Consistency:**

All rounding operations must be performed conservatively in favor of protocol solvency. Under no circumstances shall rounding result in the minting of unbacked shares or the release of excess USDC beyond what is fully collateralized. Any rounding-induced residual value must be retained within the system to preserve the 1:1 backing invariant and ensure the protocol remains overcollateralized rather than undercollateralized.

- **Accounting Correctness:**

All state variables must accurately and unambiguously reflect the true economic state of the system at all times. Internal accounting (including total supply, total assets, liabilities, and tranche balances) must remain consistent with the actual USDC held and the protocol's NAV.

State transitions must be atomic and deterministic, ensuring that no intermediate or partial updates can lead to inconsistencies. At no point shall discrepancies arise between recorded balances and actual underlying assets. Any operation that mutates state must

preserve all core invariants, including full collateral backing, correct supply accounting, and accurate distribution of gains or losses across tranches.

Relevant Properties:

- [P-03](#). Solvency
- [P-05](#). No USDC drain
- [P-20](#). Integrity of rebalance

Impact: This risk carries critical severity for the protocol. Any accounting inconsistency or breach of solvency guarantees may lead to a depegging event of MZUSD from its intended 1:1 value with USDC. Furthermore, such inconsistencies could result in a denial-of-service (DoS) condition for withdrawals, preventing users from redeeming their assets.

Oracle Usage and Correctness

Affected Components: NavOracleAutomation.sol, IssuanceVault.sol, LossManager.sol

Risk Description: Oracles are a critical dependency within the Mezzanine Protocol, serving as the primary source of truth for Net Asset Value (NAV) calculations. They directly influence the determination of yield accrual and loss realization, which in turn governs token minting, redemption logic, and tranche-level accounting.

Key Considerations:

- **Oracle Synchronization:**

The protocol relies on both the NAV oracle and the total supply oracle as joint inputs for computing yield accrual and loss realization. These values must be strictly synchronized and reflect the same point in time.

Any desynchronization between the two oracle feeds—whether due to update delays, differing reporting frequencies, or stale data—can result in incorrect yield or loss calculations. Specifically, this may lead to yield overestimation (resulting in unbacked minting) or loss miscalculation (resulting in unfair or incomplete loss socialization).

- **Oracle Usage and Freshness:**

Oracle reports must be strictly unique per epoch and reflect fresh, non-reused data. It must be impossible for multiple epochs to consume the same oracle readings or rely on stale reports.

Each oracle update must be clearly associated with a single epoch, ensuring that previously consumed data cannot be reused. Failure to enforce report freshness and uniqueness may result in double counting of yield or losses, leading to incorrect NAV updates, over-minting, or improper loss allocation.

Relevant Properties:

- [P-39](#). `performUpkeep()` or `rejectPendingNav()` clear pendings fields
- [P-40](#). Integrity of `submitNav()`

Impact: Incorrect usage of oracle data carries critical severity for the protocol. Any misuse, misinterpretation, or reliance on invalid (e.g., stale, desynchronized, or manipulated) oracle inputs may directly compromise solvency guarantees and the correctness of reward distribution.

This can lead to inaccurate NAV calculations, resulting in over-minting of unbacked tokens, under-collateralization, or improper loss realization.

ERC4626 Correctness, Reward Distribution and Loss Socialization

Affected Components: `NavOracleAutomation.sol`, `LossManager.sol`, `IssuanceVault.sol`, `MZUSD.sol`, `Tranche.sol`, `PolJuniorTranche.sol`

Risk Description: The protocol relies on oracle-reported NAV updates to trigger yield distribution and loss realization across tranches. Upon a yield event, new MZUSD is minted and allocated proportionally based on each tranche's Total Value Locked (TVL) and risk premium. Conversely, during a loss event, losses are absorbed by tranches according to a predefined priority order.

Any inconsistency, miscalculation, or improper sequencing in this process may result in incorrect reward distribution or unfair loss socialization. This includes scenarios where yield is over- or under-allocated relative to actual NAV changes, or where losses are applied out of order, partially applied, or not fully propagated across all affected tranches.

Key Considerations:

- **Manipulation of TVL to extract rewards**

The protocol distributes yield based on each tranche's Total Value Locked (TVL), making TVL a critical input for reward allocation. This creates an attack surface where adversaries may attempt to manipulate TVL—e.g., through flash deposits, short-lived capital inflows, or timing strategies around oracle updates—to disproportionately increase their share of rewards without bearing corresponding economic risk.

All protocol-imposed limits and mechanisms must reliably prevent or significantly reduce the effectiveness of TVL manipulation strategies. TVL used for reward calculations must reflect sustained economic participation rather than transient or opportunistic capital movements.

- **Front-running a reward or loss event**

Cooldown mechanisms must ensure that users cannot strategically time deposits or withdrawals to evade loss realization or front-run reward distribution events.

- **Stuck Rewards**

Virtual shares must not accumulate a material amount of unclaimable rewards. Reward distribution logic must ensure that only tranches with active stakers are eligible to receive yield allocations. The IssuanceVault shall not hold residual MZUSD, which was intended to be distributed as rewards.

- **EIP-4626 Compatibility**

The tranches inherit from the ERC-4626, so all methods must adhere to its accounting invariants and maintain a correct relationship between total assets and total shares. All state-changing functions must align with their corresponding preview functions, and all `preview*` and `max*` methods should return accurate, up-to-date results.

Relevant Properties:

- [P-19](#). `distributeYield()` at most once per epoch
- [P-22](#). Integrity of `totalAssets()`
- [P-24](#). Exchange rate only increases
- [P-26](#). `depositLimit` cannot be inflated

- [P-33](#). totalUncoveredLossMZ non decreasing
- [P-36](#). No partial loss absorption
- [P-32](#). Integrity of applyLoss
- [P-47](#). Integrity of notifyYield()
- [P-61](#). Unaccrued yield is non-increasing

Impact: Incorrect or unfair reward distribution may enable theft of rewards, escaping socialization losses or stuck funds.

Governance Actions and Access Control

Affected Components: NavOracleAutomation.sol, UmbrellaEpochFinalizer.sol, Allowlist.sol, IssuanceVault.sol, LossManager.sol, MZUSD.sol, PolJuniorTranche.sol, ReservoirUSDC.sol, Tranche.sol

Risk Description: Access control is a critical security boundary for the protocol and must strictly enforce that only authorized and trusted entities can perform privileged operations. These operations include, but are not limited to, modifying key configuration parameters, managing pause/unpause functionality, and executing upgrades.

The protocol further relies on an allowlist mechanism to restrict minting and burning of MZUSD to approved participants. This mechanism must be robust and resistant to bypass, ensuring that unauthorized actors cannot alter the total supply or otherwise interfere with core accounting invariants.

Additionally, governance and administrative changes must be applied in a predictable, forward-looking manner. State transitions resulting from such changes must not introduce retroactive effects that could alter past accounting, invalidate prior assumptions, or create inconsistencies in user balances and protocol state.

Key Considerations:

- **Access Control Correctness:**

Access control correctness is a key consideration because privileged functions directly govern the protocol's most security-critical operations, including supply changes, configuration updates, pausing, and upgrades. Any failure to correctly restrict these

actions may allow unauthorized parties to manipulate core protocol behavior, violate accounting invariants, or disrupt normal operation.

- **Governance Predictability:**

Protocol participants and integrators rely on stable, transparent, and forward-looking behavior of administrative actions. Any governance decision—such as parameter updates, upgrades, or role changes—must be applied in a deterministic and non-retroactive manner to preserve user expectations and economic fairness.

Relevant Properties:

- [P-65](#). Only allowed users can mint/redeem MZUSD
- [P-69](#). polPayout contract must not be PolJuniorTranche

Impact: Failures in access control or governance predictability carry critical severity for the protocol. Unauthorized access or privilege escalation may enable malicious actors to mint or burn MZUSD, alter key configuration parameters, or execute arbitrary upgrades, potentially leading to total loss of funds and systemic compromise.

Epoch Advancement and Automation

Affected Components: NavOracleAutomation.sol, UmbrellaEpochFinalizer.sol, IssuanceVault.sol, LossManager.sol, MZUSD.sol, PolJuniorTranche.sol, ReservoirUSDC.sol, Tranche.sol

Risk Description: Epoch advancement is a critical mechanism for the correct functioning of the protocol, as it governs the periodic finalization of accounting, including NAV updates, reward distribution, and loss realization. The system must guarantee that epochs can always be advanced and finalized without risk of failure or protocol bricking.

The integration with Chainlink Automation must be correctly implemented and robust against edge cases, ensuring that all required conditions for execution are consistently met. Automation triggers (e.g., [checkUpkeep](#) / [performUpkeep](#)) must reliably detect when an epoch is ready to advance and execute the transition without reverting or entering an inconsistent state.

Key Considerations:

- **Operational DoS due to Epoch Advancement Reverts:**

Epoch advancement logic must be designed to never enter a permanently reverting state. Any revert during the epoch transition process (e.g., during NAV updates, reward distribution, or loss application) may prevent further progression of epochs and halt core protocol functionality.

- **Chainlink Automation Correctness:**

The `checkUpkeep` and `performUpkeep` functions must be correctly designed and tightly aligned to ensure efficient and reliable automation. In particular, `checkUpkeep` must accurately determine when upkeep is required, avoiding false positives that would trigger unnecessary executions.

- **Epoch Finalization:**

Epoch finalization must only be executed after the designated epoch has fully expired, ensuring that all inputs (including oracle data and state variables) correspond to the correct accounting period. Finalization must rely on accurate, up-to-date rates to correctly compute NAV,

Relevant Properties:

- [P-07](#). Epoch ID non-decreasing
- [P-19](#). `distributeYield()` at most once per epoch
- [P-60](#). Integrity of `finalizeEpoch()`

Impact: Any flaw in epoch advancement or automation flow may lead to severe operational disruption for the protocol. If epochs cannot be advanced or finalized correctly, core system functions such as NAV updates, reward distribution, loss socialization, minting, and withdrawals may be delayed, corrupted, or fully halted, resulting in a denial-of-service (DoS) condition.

Withdrawals and Claims

Affected Components: `IssuanceVault.sol`, `MZUSD.sol`, `Tranche.sol`

Risk Description:

MZUSD Withdrawals are subject to global caps designed to ensure predictable reserve liquidity and prevent excessive outflows within a given period. These limits must be strictly

enforced across all withdrawal paths, ensuring that aggregate withdrawals never exceed the configured thresholds. When withdrawal conditions are satisfied, requests must be processed correctly and deterministically, with accurate accounting of transferred amounts and associated fees. Fee calculations must be consistent, correctly applied, and aligned with the protocol's economic design.

Tranche Withdrawals allow users to redeem tranche share tokens in exchange for MZUSD. These withdrawals are subject to enforced delays to prevent immediate entry/exit strategies and ensure fair participation across epochs. Withdrawal execution must correctly account for the difference between the request price and the final execution price, with fees applied precisely to this offset.

Key Considerations:

- **Reserved Assets:**

Upon tranche withdrawal finalization, the assets corresponding to burned shares must be explicitly reserved for the withdrawing users. These reserved amounts represent finalized liabilities of the protocol and must be segregated from active funds. Reserved assets must not be subject to subsequent loss events, reallocation, or dilution. At all times, the total reserved amount must be fully backed by the protocol's MZUSD balance, ensuring that all user claims remain solvent and immediately claimable.

- **Fee Accounting:**

Fees must be accurately calculated, consistently applied, and sourced from the correct state variables or oracle inputs. The fee logic must be deterministic and resistant to manipulation across all execution paths. Users must not be able to reduce or bypass their fee exposure through timing strategies, partial executions or rounding inconsistencies.

- **Withdrawal Limit and Whitelisting:**

Withdrawal limits must be strictly and globally enforced, ensuring that total withdrawals cannot exceed predefined caps under any circumstances. Only explicitly allowlisted users must be permitted to execute withdrawals.

Relevant Properties:

- [P-25](#). userSharesByEpoch tracked correctly

- [P-27](#). No double claiming
- [P-60](#). Integrity of finalizeEpoch()
- [P-30](#). Integrity of requestWithdraw
- [P-11](#). Rounding in MZUSD always favours the protocol

Impact: Failures in withdrawal handling, limit enforcement, or fee accounting may lead to critical protocol risks. In particular, incorrect enforcement of withdrawal constraints or state inconsistencies may result in a denial-of-service (DoS) condition, preventing users from executing or claiming withdrawals and effectively locking funds within the system.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	5	5	2
Low	10	10	4
Informational	8	8	5
Total	23	23	11

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	applyLoss is going to double count the uncoveredMZ within totalUncoveredLossMZ	Medium	Fixed
M-02	Some tranches may be able to escape the loss using instant redemptions	Medium	Acknowledged
M-03	Tranche deposit limit bypass due to using live totalSupply can be used to manipulate yield distribution	Medium	Acknowledged
M-04	Same report usage for two different epochs can result in double counting loss/yield	Medium	Partially fixed
M-05	Insufficient validation of oracles synchronization	Medium	Acknowledged
L-01	Missing implementation to handle the totalUncoveredLossMZ	Low	Acknowledged
L-02	maxWithdraw and maxRedeem do not comply with eip4626	Low	Partially fixed
L-03	Tranche virtual shares may accrue excessive yield, which may eventually be stuck	Low	Fixed
L-04	Tranche inflation attack via truncation	Low	Acknowledged

L-05	setOracles() could be used under timelock to increase predictability	Low	Acknowledged
L-06	Retroactive admin withdraw fee change could disturb claims	Low	Acknowledged
L-07	checkUpkeep will return an incorrect output in multiple cases	Low	Fixed
L-08	Oracle design shall be improved in order to preserve solvency	Low	Acknowledged
L-09	All liquidity can be instantly withdrawn from the reservoir by allowed entities	Low	Acknowledged
L-10	applyLoss may not restore solvency in an edge case	Low	Fixed
I-01	Users can invoke claim for other tranches to increase their loss	Informational	Acknowledged
I-02	Unnecessary _withdraw() validation	Informational	Fixed
I-03	Allowlist contract does not call _disableInitializers()	Informational	Fixed
I-04	IssuanceVault fee configuration can be set incorrectly	Informational	Partially fixed
I-05	withdrawnTo mapping may be inconsistent	Informational	Acknowledged

I-06	setPolPayout shall not allow setting the payout address to address(this)	Informational	Fixed
I-07	Missing implementation to remove a tranche could cause DOS	Informational	Acknowledged
I-08	Fee is rounded down, making it profitable to split withdrawals	Informational	Fixed

Medium Severity Issues

M-01: applyLoss is going to double count the uncoveredMZ within totalUncoveredLossMZ

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: contracts/LossManager.sol	Status: Fixed	

Description:

The `NavOracleAutomation` is going to submit and apply the NAV once per epoch. If there is a loss event, the loss is going to be consumed from the tranches.

If the loss exceeds the stake within the tranches, then `totalUncoveredLossMZ` is going to be increased.

However the `totalUncoveredLossMZ` may actually count the same loss multiple times. For example, if the NAV is constant across 2 different epochs and there is no stake within the tranches, with `totalSupply` less than the NAV, the following will be added to the `totalUncoveredLossMZ` multiple times: $NAV - totalSupply$

As a result, the same loss will be double counted, causing the `totalUncoveredLossMZ` to be incorrect.

Recommendations:

If there is a loss, do not increase the `totalUncoveredLossMZ`, but consider overriding it, so it represents the latest loss.

Customer Response: Fixed in [9397e5cb3a](#)

Fix Review: Fix confirmed

M-02: Some tranches may be able to escape the loss using instant redemptions

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: contracts/automation/NavOracleAutomation.sol	Status: Acknowledged	

Description:

The oracle has introduced a veto period, which has to expire before applying the oracle results to the tranches. However, some tranches allow for instant redemptions, which may be used by active users to escape the loss socialization, shifting the loss to other tranches.

Recommendations:

Consider always enforcing a withdrawal delay on all tranches, so frontrunning slashing cannot be executed.

Customer Response: Acknowledged

"Only Senior tranche will have instant withdrawal. This Tranche has the lowest risk and loss is applied to it in the last turn. Therefore, cases where loss touches Senior tranche are rare and require to fully exhaust Junior and Mezzanine tranche. Furthermore, it is likely that with such a great loss, the protocol will be paused until further recovery."

M-03: Tranche deposit limit bypass due to using live totalSupply can be used to manipulate yield distribution

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: contracts/Tranche.sol	Status: Acknowledged	Violated property: P-26 Deposit limit cannot be inflated

Description:

The `_computeDepositLimit` will perform the following calculation in order to determine the deposit limit:

```
uint256 assetTotalSupply = IERC20(asset()).totalSupply();
    uint256 trancheAssets = totalAssets(); // virtual TVL with streaming
    uint256 maxTotalByTvl = (assetTotalSupply * maxDepositTvlBps) /
BASIS_POINTS;
```

However, since the limit is calculated based on the live total supply of MZUSD, any allowlisted user will be able to bypass it by inflating the total supply of MZUSD.

This could cause serious issues with yield distribution, which heavily relies on the deposit limit to preserve the TVL allocation across tranches.

Since the deposit limit can be bypassed by any allowlisted user, this could result in a compromised yield round.

For example:

An allowlisted user front-runs the `_allocateYieldToTranches` call with the following sequence:

1. Mint a large amount of MZUSD to increase the limit
2. Deposit a portion of the minted amount in the tranche.
3. Redeem the leftover unused MZUSD

As a result, the attacker will be able to allocate more yield to a certain tranche.

After that they can redeem at any point in the future the tranche deposit.

Recommendations:

The following root-causes shall be fixed:

- The deposit limit using live totalShares can be bypassed by allowlisted users
- The yield distribution using live totalAssets can be abused by some users, to manipulate tranche yield distribution

Customer Response:

Acknowledged

"The deposit limit has a 2-layer protection: as the total supply grows, the cap stops at a predefined value. Thus, the users can't affect the cap more than a certain limit. The limits are designed to ensure a certain yield distribution (Ideally we want all the tranches to be filled at their maximum capacity). Additionally, Mezzanine and Junior tranches have a delayed withdrawal and the attacker will have to keep the funds in the tranches for some epochs before withdrawing. Also, if we detect someone manipulating the system, we will remove such a user from allowlist."

M-04: Same report usage for two different epochs can result in double counting loss/yield

Severity: Medium	Impact: Medium	Likelihood: Low
Files: contracts/automation/NavOracleAutomation.sol contracts/IssuanceVault.sol	Status: Partially fixed	Violated property: P-56 : NAV value returned from oracle cannot be used in two epochs

Description:

When `submitNav()` is called, it is going to verify that there is no pending Nav:

```
if (hasPendingNav()) revert PendingNavExists();
```

After that, the fresh answers are read from the `navOracle` and the `totalSupplyOracle`, performing the following staleness checks:

```
if (block.timestamp - navUpdatedAt > uint256(maxAnswerStaleness)) {  
    revert StaleOracleAnswer(address(navOracle_), navUpdatedAt,  
block.timestamp, maxAnswerStaleness);  
}
```

Under specific conditions, this staleness check can be insufficient.

Consider the following scenario:

1. `maxAnswerStaleness` = 1 hours (`DEFAULT_MAX_ANSWER_STALENESS`),
`vetoPeriod` = 0
`epochDuration` = 1 hours (`MIN_EPOCH_DURATION_LIMIT`)
2. At `T = 60` minutes, `submitNav` is invoked starting epoch X. It is going to use `reportX`, which was reported at `T=60`
3. At `T = 120` minutes, `submitNav` is invoked again, using `reportX` from the oracle reports:

1. The staleness check is going to pass:
`if (block.timestamp - navUpdatedAt > uint256(maxAnswerStaleness))`

2. The epoch can be updated:

As a result, the same report from the oracles can be used twice under the allowed configuration of the vault and the oracle automation, potentially allowing double counting losses.

Recommendations:

Consider changing the `DEFAULT_MAX_ANSWER_STALENESS` to prevent such overlaps and enforcing additional limitations on the setter, by comparing the allowed staleness period with the `vetoPeriod` and the epoch duration.

Customer Response:

Fixed in [3ba079e574](#)

Fix **Review:** Partially fixed. `IssuanceVault::setEpochConfig()` may still cause `maxAnswerStaleness >= epochDuration`.

M-05: Insufficient validation of oracles synchronization

Severity: Medium	Impact: High	Likelihood: Low
Files: contracts/automation/NavOracleAutomation.sol	Status: Acknowledged	

Note: The severity of this issue was originally assessed as High. However, the protocol team clarified that the veto mechanism will be actively utilized to prevent incorrect or desynchronized oracle readings from being accepted. Therefore, the likelihood has been downgraded to Low and the severity to Medium.

Description: In order to accurately determine the yield/loss from the oracles, it is important that both oracles were reading from the same state – within the same transaction. This means that the `totalSupply` of MZUSD when the NAV oracle reported the USDC must be equal to the `totalSupply` reported by the `totalSupplyOracle`.

However, this relationship is not reliably validated. Instead, the `updatedAt` timestamp is used:

```
// Both oracles should always be updated in the same transaction
if (navUpdatedAt != supplyUpdatedAt) revert OraclesNotInSync();
```

The `updatedAt` timestamp represents the moment the oracles were updated. Therefore, it is possible that oracles did not accurately represent the unbacked MZUSD or the excessive USDC yield. Both oracles may be updated within the same timestamp, but represent a different state.

Recommendations: Consider enforcing a reliable solution to properly validate that the oracles were reading from the same state.

Customer's response: Acknowledged

"We rely on Accountable which monitors the underlying balances of the protocol to report NAV and Total Supply at which the NAV is calculated and the Redstone oracle. The Redstone team has assured that their solution retrieves both values from Accountable which correspond to each other and pushes them into two separate oracles in the same transactions. Thus, they

ensure that two values correspond to each other. Since we have the veto period added, we would catch an incongruence between the submitted values."

Low Severity Issues

L-01: Missing implementation to handle the totalUncoveredLossMZ

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/LossManager.sol	Status: Acknowledged	

Description:

The `totalUncoveredLossMZ` is only increased when there is a loss event that exceeds the staked MZUSD within the tranches subject to a loss.

However, there is no implemented way in the code to handle the `totalUncoveredLossMZ` or decrease it in any way.

Recommendations:

Consider adding a way to handle the `totalUncoveredLossMZ`.

Customer Response:

Acknowledged

"The variable currently accumulates all uncovered losses and serves as a view metric only. It doesn't affect any yield/loss distributions in the system. We might add any usage for it later including clearing its state."

Fix Review:

Note that before using this variable, it is crucial to fix [M-01](#) from the report.

L-02: maxWithdraw and maxRedeem do not comply with eip4626

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/Tranche.sol	Status: Partially fixed	

Description:

The `Tranche` contract inherits from the `ERC4626Upgradeable` which implements the `maxWithdraw` and `maxRedeem` functions. However, those functions are not overridden to comply with the standard: **MUST factor in both global and user-specific limits, like if withdrawals are entirely disabled (even temporarily) it MUST return 0.** For tranches with disabled `directWithdrawEnabled` this would cause those max functions to return an incorrect amount. As a result, pausing may also be bypassed using the `withdraw` and `redeem` functions directly.

Additionally, for the instant `withdraw`/`redeem` cases, both function do not factor in the following limit:

```
if (shares == totalSupply() - balanceOf(VIRTUAL_SHARES_RECEIVER) &&
    _unaccruedYield() != 0)
    revert FullWithdrawNotAllowed();
```

Recommendations:

Consider overriding the `max withdraw` and `redeem` functions to make them compatible with eip4626.

Customer Response:

Fixed in [1c1b91115a](#)

Fix Review: Partially fixed

The withdrawals now account for the paused withdrawal flag. However, if the MZUSD token has enabled the transfer pausing, the max functions may still return an incorrect amount.

L-03: Tranche virtual shares may accrue excessive yield, which may eventually be stuck

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/Tranche.sol	Status: Fixed	

Description:

The `_allocateYieldToTranches()` function is intended to skip tranches without any TVL:

```
if (trancheTvl == 0) {  
    ++i;  
    continue;  
}
```

However, it is possible for a tranche to have some TVL, without any `totalSupply`. This is going to cause virtual shares to accrue excessive yield, which is eventually going to be stuck into the tranche.

Recommendations:

Consider skipping tranches which do not have any supply.

Customer Response:

Fixed in [4d7e644c9e](#)

Fix Review:

Fix confirmed.

L-04: Tranche inflation attack via truncation

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/Tranche.sol#L552	Status: Acknowledged	

Description:

The decimals offset has been set to 3 in order to limit the first depositor attack:

```
function _decimalsOffset() internal pure override returns (uint8) {  
    return 3;  
}
```

However, this limitation alone is not fully protecting against such attacks. The first depositor can still inflate the shares of the tranche, causing truncation and precision losses for other depositors.

Recommendations:

It is recommended to make an initial deposit for the tranche.

Customer Response:

Acknowledged.

"We already have virtual shares minting in the deployment script. will ensure that they are minted for every Tranche."

L-05: setOracles() could be used under timelock to increase predictability

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/automation/NavOracleAutomation.sol#L153	Status: Acknowledged	

Description:

Oracles are a critical part of the protocol, directly responsible for the loss and yield that users will accrue. Changing the oracle should be predictable by users.

Recommendations:

Consider using a timelock for changing critical components in the codebase. Alternatively, a fallback oracle can be used, in order to switch to it when the main oracles are deemed incorrect.

Customer Response:

Acknowledged

"Initially, Admin role will be granted to the protocol's multisig. We will add timelock for all admin functions in future phases."

L-06: Retroactive admin withdraw fee change could disturb claims

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/Tranche.sol	Status: Acknowledged	

Description:

The `unstakeYieldReductionBps` can be changed anytime allowing for retroactive changes by the admin on existing claims. This creates an unpredictable environment for users. If a user requests a withdrawal in epoch1 and the withdrawal is completed in epoch3, changing the `unstakeYieldReductionBps` during epoch 2 or later, would directly impact the withdrawal initiated in epoch1. Furthermore, the fee may be changed after the withdrawal is finalized and still affect the claims.

Recommendations:

Consider snapshotting the fee upon requesting the withdrawal.

Customer Response:

Acknowledged.

"This is a known issue, will implement a fix in the future versions."

L-07: checkUpkeep will return an incorrect output in multiple cases

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/automation/NavOracleAutomation.sol contracts/automation/UmbrellaEpochFinalizer.sol	Status: Fixed	Violated property: P-57 : Integrity of checkUpkeep()

Description:

The `checkUpkeep` function is used by the Chainlink automation in order to determine whether to call `performUpkeep`. If this function repeatedly returns `true` when the actual `performUpkeep` function is reverting, it is going to cause losses for the protocol, due to attempting to call `performUpkeep` multiple times unsuccessfully. This occurs in multiple instances:

1. In `UmbrellaEpochFinalizer`, when the `PROTOCOL` pause flag is on, the tranche `finalize` function is going to revert. However, `UmbrellaEpochFinalizer::checkUpkeep` does not factor in that case.
2. The `NavOracleAutomation::checkUpkeep()` function must return false when all the tranches do not have any TVL:
3. `NavOracleAutomation::checkUpkeep()` will not return false when the `PROTOCOL` pause is enabled

Recommendations:

4. Consider always returning false for `checkUpkeep` when `performUpkeep` is going to revert.

Customer Response:

Fixed in [7158242e9f](#) and [34299deef9](#)

Fix Review: Fix confirmed.

L-08: Oracle design shall be improved in order to preserve solvency

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/automation/NavOracleAutomation.sol	Status: Acknowledged	

Description:

The current design allows a malicious NAV oracle to potentially apply an incorrect NAV (assuming there is no veto), resulting in minting of unbacked MZUSD.

Breaking the solvency of the protocol could have catastrophic consequences on all integrations of the MZUSD token.

There are additional limitation that can be enforced in order to prevent the NAV oracle from minting a large amount of unbacked shares:

- Enforcing a bucked limit on the earnings that the NAV oracle can report.
- Using a dual oracle design to determine if the oracle report is off.
- Adding a minimum veto period, so compromised oracles can be stopped from introducing an incorrect NAV.

Recommendations:

Consider implementing the additional limitations in [NavOracleAutomation](#) to reliably preserve the solvency.

Customer Response:

Acknowledged

“Will implement a deviation validation where the result from the primary oracle will be checked against one or multiple secondary oracles. If deviation exceeds the allowed threshold, such NAV won't be submitted. This change will be introduced in Phase 1.5”

L-09: All liquidity can be instantly withdrawn from the reservoir by allowed entities

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/ReservoirUSDC.sol	Status: Acknowledged	

Description: The `ReservoirUSDC::withdraw()` function is going to enable allowed entities to withdraw any amount of USDC from the reservoir. However, this introduces a centralization risk, which could potentially enable an attacker to steal the USDC backing from the vault.

Recommendation: Consider enforcing bucket limits to prevent instant large withdrawals from the reservoir. Alternatively, consider adding an allowlist, which can add allowed recipients under timelock.

Customer Response:

Acknowledged

"Will implement a withdrawal allowlist. The admin will be able to set addresses in the allowlist: only strategy vaults will be included so that Curators can't withdraw funds directly to their or any other wallets. This change will be introduced in Phase 1.5"

L-10: applyLoss may not restore solvency in an edge case

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/automation/NavOracleAutomation.sol	Status: Fixed	Violated property: P-32 : Integrity of applyLoss()

Description: The `NavOracleAutomation::_deltaUSDC` function is going to calculate the amount of required USDC backing by performing the following:

```
uint256 totalSupplyUSDC = totalSupplyMZ / SCALE;
```

However, this is going to round down the amount of USDC required in order to facilitate the solvency. As a result, the loss is going to be underestimated, leaving up to 1e12MZUSD unbacked.

Recommendation: Consider rounding the `totalSupplyUSDC` up, in order to fully restore the collateralization.

Customer Response:

Fixed in [3cd07569a4](#)

Fix Review:

Fix confirmed

Informational Severity Issues

I-01: Users can invoke claim for other tranches to increase their loss

Description:

Loss is distributed according to the TVL, which excludes the reserved assets.

However, a portion of the reserved assets is released into the `totalAssets` upon invoking the `claim` function.

As a result, users from other tranches may process claims before a loss event, to impact the TVL and consume more of the loss.

The claim function may also be subject to other racing conditions, since it is instantly increasing the share value for the remaining liquidity providers. For example, a similar issue can occur by front-running the reward distribution by processing all the claims in order to release the `unstakeYieldReductionBps`.

Recommendations:

The fix of the issue is non-trivial. Consider acknowledging this issue.

Customer Response:

Acknowledged

"Will think of ways to fix this issue in future phases. Forcing the claim is necessary to exclude stale funds from Tranche's balance if users do not claim. And the issue is possible only if there are pending claims which the users decided not to claim."

I-02: Unnecessary `_withdraw()` validation

Files:

[contracts/Tranche.sol](#)

Description:

The `_withdraw()` function is going to perform the following validation:

```
if (shares == totalSupply() - balanceOf(VIRTUAL_SHARES_RECEIVER) &&
_unaccruedYield() != 0) {
    revert FullWithdrawNotAllowed();
}
```

Its purpose is to prevent full amount withdrawals while there is `_unaccruedYield()`, so all yield can be distributed to users.

However, virtual shares are still accruing yield and this validation does not prevent it. It may only force the last user to leave 1 wei, so they can receive 1/101 of the accrued yield.

Furthermore, the 2-step withdrawal path, when the direct withdrawals are disabled does not perform such validation.

Recommendations:

Consider removing the unnecessary check.

Customer Response:

Fixed in [8a9d5bc2cf](#)

Fix Review:

Fix confirmed

I-03: Allowlist contract does not call `_disableInitializers()`

Description:

The Allowlist contract is an implementation, intended to be used by a proxy contract. However, it does not invoke `_disableInitializers` in the constructor, allowing for anyone to call `initialize` on the implementation itself.

Recommendations:

Consider invoking `_disableInitializers` in the constructor of Allowlist.

Customer Response:

Fixed in [8e6531223e](#)

Fix Review:

Fix confirmed

I-04: IssuanceVault fee configuration can be set incorrectly

Description:

The `performanceFeeBps` and `protocolFeeBps` can be set by the owner using the `setFeeRates()` or within the `initialize()` function. However, setting a positive fee does not check if the `performanceFeeReceiver` and the `protocolFeeReceiver` are non-zero.

Violated property: [P-21](#): Valid state conditions and helper invariants

Recommendations:

Consider validating that a valid receiver has been set, before setting a non-zero fee.

Customer Response:

Fixed in [40b621a67e](#)

Fix Review:

Partially fixed. Incorrect initialization may still lead to that incomplete state.

I-05: withdrawnTo mapping may be inconsistent

Description:

The `withdrawnTo[]` mapping is going to store the balances that withdrawers receive from the `Reservoir`. However, if a user passes in the `Reservoir` address as a `to` parameter, this could cause the vault to display an amount that was not actually withdrawn, which may potentially cause inconsistencies in future versions or in off-chain accounting.

Violated property: [P-67](#): Integrity of ReservoirUSDC accounting

Recommendations:

Consider not allowing the `to` parameter to be equal to `address(this)`.

Customer Response:

Acknowledged

"Will implement an allowlist of withdrawal destination addresses in the future phase."

I-06: setPolPayout shall not allow setting the payout address to address(this)

Description:

In PolJuniorTranche::setPolPayout can be set to address(this):

```
function setPolPayout(address payout) external restricted
whenNotPaused(PauseCategory.ADMIN) {
    if (payout == address(0)) revert ZeroAddress();
    polPayout = payout;
    emit PolPayoutUpdated(payout);
}
```

As a result, this may cause issues with the yield distribution, being able to emit the same event multiple times:

```
function notifyYield(
    uint256 yieldAmount
) external onlyVault whenNotPaused(PauseCategory.PROTOCOL) {
    if (yieldAmount == 0) revert ZeroYieldPassed();
    mzusd.transfer(polPayout, yieldAmount);
    emit PolPayoutNotified(yieldAmount);
}
```

Violated property: [P-67](#): Payout address must not be PolJuniorTranche contract

Recommendations:

Consider not allowing the `payout` parameter to be equal to `address(this)`.



Customer Response:

Fixed in [e63b4216e2](#)

Fix Review:

Fix confirmed

I-07: Missing implementation to remove a tranche could cause DOS

Description:

All admin functions within the issuance vault can only increase the tranches array within the IssuanceVault. However, there is no implementation that allows removing a tranche from the array.

Currently, `setTrancheConfig` allows the admin to mark a tranche as disabled. However, a flawed tranche implementation may DOS the epoch advancement even if the tranche is marked as disabled.

For example, if a tranche in the array is disabled, but it is reverting all its calls, it may DOS the yield distribution and the loss socialization:

```
function _allocateYieldToTranches(uint256 allocatableMZUSD) internal returns
(uint256 actuallyAllocated) {
```

```
    ...
```

```
        for (uint256 i; i < trancheCount; ++i) {
```

```
            t = tranches[i];
```

```
            trancheTvl = ITranche(t.tranche).totalAssets(); //a reverting
tranche, will prevent yield distribution
```

```
function _applyLossAcrossOrder(uint256 requestedMZ) internal returns (uint256
burnedMZ, uint256 uncoveredMZ) {
```

```
    ...
```

```
        if (i + 1 < orderLen && t1 != address(0) && order[i + 1] == t1) {
```

```
            uint256 tvl0 = ITranche(t0).totalAssets();// a reverting tranche
can prevent loss socialization
```

Recommendations:

Consider not performing any external calls to disabled tranches.

Customer Response:

Acknowledged

"On the start, 3 tranches will be registered and new tranches won't be added soon. The logic of tranche removal will be added later for cases where the list of tranches might be updated frequently. Currently, disabling a tranche means it is excluded from yield distribution and shouldn't prevent a view call such as totalAssets."

I-08: Fee is rounded down, making it profitable to split withdrawals

Description:

In MZUSD the fee is calculated using the following formula:

```
uint256 feeUSDC = (usdcOutGross * withdrawalFeeBps) / BPS;
```

However, the feeUSDC is rounded down, which makes it profitable for users to split withdrawals to avoid paying fees.

The impact of this issue is negligible, as such attacks are not profitable due to gas costs.

Violated property: [P-11](#): Rounding in MZUSD always favours the protocol

Recommendations:

Consider rounding the fee up, or acknowledging the issue.

Customer Response:

Fixed in [079f31bcb9](#)

Fix Review:

Fix confirmed

Formal Verification

Verification Methodology

We performed verification of the **Mezzanine Protocol** protocol using the Certora Prover which uses Satisfiability Modulo Theories (SMT) solving. In short, the Certora verification tool works by compiling formal specifications written in the [Certora Verification Language \(CVL\)](#) and **Mezzanine Protocol** 's implementation source code written in Solidity.

More information about Certora's tooling can be found in the [Certora Technology Whitepaper](#).

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce over-approximations (replacing real computations with broader ranges of values) and under-approximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.

Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the contract's constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions

- We verify the following contracts: **MZUSD**, **IssuanceVault**, **ReservoirUSDC**, **LossManager**, **PolJuniorTranche**, **NavOracleAutomation**, and **PauseManager**. For some rules we use harness variants, i.e. contracts that are derived from the originals and add some additional methods (**IssuanceVaultHarness**, **NavOracleAutomationHarness**, **TrancheHarness**). These expose additional view functions and simplify internal flows without altering the verified behaviour.
- The underlying **USDC** asset is modelled by a **MockUSDC** contract (6 decimals, standard OpenZeppelin ERC20).
- The Chainlink NAV and total-supply oracles are modelled by mock feeds (**MockProtocolNavFeed**, **MockTotalSupplyOracle**)
- The external sanctions list is likewise modelled by **MockSanctionsList**.
- The OpenZeppelin **Math.mulDiv(x, y, d, rounding)** routine is replaced by our own **mulDivUpSummary** / **mulDivDownSummary** implementations in CVL, which are functionally equivalent to the originals but better suited for Certora Prover.
- For some functions like **ERC1967Utils.upgradeToAndCall** we consider that they can change the storage arbitrarily. Since it is an overapproximation it does not weaken the guarantees by formal verification.
- Solidity sources are compiled with solc 0.8.30 using the via-IR pipeline.

General Limitations

- The protocol supports an arbitrary number of tranches, but the verification instantiates at most three tranches that are an arbitrary combination of harnesses (`TrancheA`, `TrancheB`, `TrancheC`) and `PolJuniorTranche`, all sharing the same `MZUSD` asset.
- Loops are inherently difficult for formal verification. We handle them by unrolling a fixed number of times: `loop_iter = 3` for the global/cross-contract runs and `loop_iter = 1–2` for single-contract runs. This bounds the number of registered tranches and the length of tranche-iteration loops.
- We assume decimals of `MZUSD` are `18` and decimals of `Tranche` are `21`.
- Some parametric rules filter out entry points that are only reachable through the protocol's internal flow and would trivially fail when called in isolation: `MZUSD.mint` (only reachable from `IssuanceVault.distributeYield`), `IssuanceVault.withdraw` (only reachable from `MZUSD.requestWithdrawal` after burn), and `IssuanceVault.distributeYield` (only reachable from `NavOracleAutomation._applyPending`).
- We assume that the contracts in the scene are correctly linked, e.g. `Tranche.asset()` is `MZUSD`, `IssuanceVault.lossManager` is `LossManager`, etc. Due to this assumption, some setter methods are no longer applicable as they require that the value is not set already. (E.g. `IssuanceVault.setLossManager`). We don't consider these methods when proving invariants and parametric rules or prove them with slightly different settings.

Formal Verification Properties Overview

Global

ID	Title	Description	Status
P-03	Solvency	MZUSD <code>totalSupply</code> is always backed by at least an equivalent amount of USDC.	Verified
P-05	No USDC drain	USDC can only leave the system via legitimate user operations.	Verified
P-06	Integrity of <code>distributeYield()</code>	<code>distributeYield()</code> increases MZUSD <code>totalSupply</code> correctly.	Verified
P-07	Epoch ID non-decreasing	Epoch ID is strictly monotonically increasing across all epoch-advancing operations and non-decreasing for the rest of operations	Verified

IssuanceVault

P-18	Sum of Tranche BPS values is equal to 100%	The sum of the reward and risk premium BPS values for all tranches sums to 10000 BPS	Verified
P-19	<code>distributeYield()</code> at most once per epoch	<code>distributeYield()</code> can be called at most once per epoch	Verified
P-20	Integrity of <code>rebalance()</code>	Total USDC liquidity (vault idle USDC + Reservoir USDC) is conserved across any sequence of <code>rebalance()</code>	Verified

P-21	Consistency of System State and Helping Invariants	Data in <code>IssuanceVault.tranches[]</code> and other fields are always in consistent state.	Verified after fix
----------------------	--	--	--------------------

Tranche

P-22	Integrity of <code>totalAssets()</code>	<code>totalAssets() == MZUSD.balanceOf(tranche) - _unaccruedYield() - reservedAssets</code>	Verified
P-23	Integrity of Tranche <code>requestWithdrawal()</code>	In direct mode shares should immediately redeem and return MZUSD. When it is not, the shares should be transferred to the Tranche contract.	Verified
P-24	Exchange rate only increases	<code>pricePerShare()</code> can only increase or stay the same under normal conditions (no loss events)	Verified
P-25	<code>userSharesByEpoch()</code> tracked correctly	Sum of <code>userSharesByEpoch[user][epochId]</code> for all users is equal to <code>umbrellaSharesByEpoch[epochId]</code>	Verified
P-26	<code>depositLimit</code> cannot be inflated	<code>Tranche.maxDeposit()</code> is not affected by user operations.	Violated
P-27	No double claiming	A user can <code>claim()</code> for a given epochId at most once and <code>finalizedEpochs[epochId]</code> can be set only once	Verified
P-59	Tranche ERC4626 compatibility	The Tranche contract must be compliant with the ERC4626 standard	Verified after fix

P-60	Integrity of <code>finalizeEpoch()</code>	(i) <code>finalizeEpoch()</code> will allocate the correct amount of MZUSD according <code>convertToAssets</code> (ii) epoch can be finalized only after a certain period.	Verified
P-61	Unaccrued yield is non-increasing	Unaccrued yield shall be non-increasing during the epoch..	Verified
P-66	Anyone can call deposit, withdraw, claim	Anyone can interact with the tranche.	Verified

MZUSD

P-10	No MZUSD token loss	A user's MZUSD balance can decrease if-and-only-if their USDC increases	Verified
P-11	Rounding in MZUSD always favours the protocol	Rounding in favour of the protocol, i.e., <code>deposit</code> -> <code>withdraw</code> -> <code>deposit</code> cannot gain	Verified after fix
P-14	Integrity of deposit	For every <code>deposit()</code> , exactly <code>amountUSDC * SCALE</code> MZUSD is minted to the caller and exactly the <code>amountUSDC</code> is transferred to the vault.	Verified
P-15	USDC used in window is never greater than global redemption cap	<code>windowUsedUSDC</code> is never greater than <code>globalRedemptionCapUSDC</code>	Verified
P-16	<code>windowStartTimestamp</code> can only increase	<code>windowStartTimestamp</code> is never in the future	Verified

P-30	Integrity of <code>requestWithdrawal</code>	<code>requestWithdrawal(amountMZ)</code> burns exactly $\text{floor}(\text{amountMZ} / \text{SCALE}) * \text{SCALE}$ — never more than <code>amountMZ</code>	Verified
P-65	Only allowed users can <code>mint/redeem</code> MZUSD	Authorization check for <code>mint</code> and <code>redeem</code>	Verified

LossManager

P-32	Integrity of <code>applyLoss()</code>	<code>applyLoss()</code> burns an exact amount of MZUSD and makes the system solvent again. The loss that was not covered by tranches is correctly reflected by <code>totalUncoveredLossMZ</code>	Verified after fix
P-36	No partial loss absorption	If <code>usdcLoss > mzusd.totalSupply / SCALE</code> , <code>applyLoss()</code> reverts rather than applying the loss to the current supply.	Verified

NavOracleAutomation

P-38	Integrity of <code>NavOracleAutomation.performUpkeep</code>	When <code>pendingYield</code> is true <code>performUpkeep</code> calls <code>distributeYield</code> and <code>applyLoss</code> otherwise	Verified
P-39	<code>performUpkeep()</code> or <code>rejectPendingNav()</code> clear <code>pendings</code> fields	After <code>performUpkeep()</code> or <code>rejectPendingNav()</code> , <code>hasPendingNav() == false</code> and all pending fields are reset to zero	Verified
P-40	Integrity of <code>submitNav()</code>	(i) <code>submitNav()</code> only accepts oracle answers that are no older than <code>maxAnswerStaleness</code> and whose NAV-feed and total-supply-feed <code>updatedAt</code> timestamps are in sync	Verified

		(ii) when <code>vetoPeriod == 0</code> the new NAV is applied in-line, otherwise it is queued with <code>pendingReadyAt = submittedAt + vetoPeriod</code> and that snapshot is immune to subsequent <code>setVetoPeriod</code> changes.	
P-56	NAV value returned from oracle cannot be used in two epochs	An oracle report with the same timestamp cannot be used for two consecutive epochs	Verified after fix
P-57	Integrity of <code>checkUpkeep()</code>	<code>checkUpkeep()</code> must return false (or revert) if <code>performUpkeep()</code> is going to revert	Verified after fix

ReservoirUSDC

P-44	Integrity of public methods	<code>deposit()</code> the Reservoir's USDC balance increases by exactly the deposited amount and <code>withdraw()</code> transfers exactly <code>amount</code> USDC to <code>to</code> .	Verified
P-67	Integrity of ReservoirUsdc accounting	Correct accounting in <code>depositedBy</code> and <code>withdrawnTo</code>	Violated

PolJuniorTranche

P-47	Integrity of <code>notifyYield()</code>	<code>notifyYield()</code> transfers exactly <code>amount</code> MZUSD from this contract to <code>polPayout</code> in the same call — no intermediate streaming, escrow, or partial forwarding .	Verified
P-69	<code>polPayout</code> contract must not be <code>PolJuniorTranche</code>	<code>PolJuniorTranche.polPayout</code> is not the <code>PolJuniorTranche</code> contract itself.	Verified after fix

PauseManager

P-49	Idempotence of <code>PauseManager</code>	Pausing an already-paused category is idempotent (does not revert, state remains paused). Unpausing an already-unpaused category is idempotent.	Verified
----------------------	--	--	----------

Detailed Properties

Global Invariants

P-03. Solvency			
Status: Verified			
Rule Name	Status	Description	Link to rule report
p3a_fullSolvency	Verified	<i>MZUSD totalSupply is always backed by at least an equivalent amount of USDC expressed via the oracle deployed NAV:</i> <pre>mzUSD.totalSupply() <= getDeployedNav(e) * SCALE.</pre> <i>Filters out MZUSD.mint (only reachable from IssuanceVault.distributeYield), IssuanceVault.withdraw (only reachable from MZUSD.requestWithdrawal after burn), and IssuanceVault.distributeYield (only reachable from NavOracleAutomation._applyPending).</i> <i>We assume navOracle.pendingNavOutsideChange < 0 =></i> <pre>-navOracle.pendingNavOutsideChange <= navOracle.NAVOutside</pre> <i>For mzUSD.deposit(), we assume that the message sender is not the vault or the reservoir.</i> <i>For reservoir.withdraw(), we assume that it is withdrawn to the vault.</i>	Report
p3b_fullSolvency OracleYield	Verified	<i>Solvency bound mzUSD.totalSupply() <= getDeployedNav(e) * SCALE. proved for NavOracleAutomation methods that apply a pending USDC delta (submitNav, performUpkeep).</i> <i>We assume navOracle.pendingNavOutsideChange < 0 =></i> <pre>-navOracle.pendingNavOutsideChange <= navOracle.NAVOutside;</pre> <i>to reasonably bound the delta reported by the oracle.</i>	Report
p3c_fullSolvency	Verified	<i>Solvency bound holds for methods run in configurations where some cross-contract links are not wired (noLinkingMethods filter). Basically these are setters methods.</i>	Report

contractNeverCallsSelf	Verified	Proves that the assumption <code>msg.sender != mzusd</code> in <code>noSelfCalls(mzusd)</code> is safe by showing that no external function on the contract causes it to call back into itself.	Report
------------------------	----------	---	------------------------

P-05. No USDC drain

Status: Verified

Rule Name	Status	Description	Link to rule report
p5a_noUSDCDrain	Verified	Excludes oracle-yield and <code>noLinkingMethods</code> that are proven in the next two rules	Report
p5c_noUSDCDrain_setters	Verified	Proved for setters methods	Report

P-06: Integrity of distributeYield

Status: Verified

Rule Name	Status	Description	Link to rule report
p6d_distributeYield	Verified	<code>distributeYield(usdcReported)</code> increases <code>MZUSD totalSupply</code> by exactly <code>usdcReported * MZUSD_SCALE</code> and increases the sum of balances of Tranches by the same amount. In case that one of the tranches is <code>Pu1JunTranche</code> we check it for case that <code>polPayout</code> is not <code>IssuanceVault</code> . We require <code>performanceFeeReceiver != vault</code> and <code>protocolFeeReceiver != vault</code>	Report

P-07: Epoch ID non-decreasing

Status: Verified

Rule Name	Status	Description	Link to rule report
p7a_advancingEpoch	Verified	Methods classified as epoch-advancing (<code>distributeYield()</code> , <code>advanceEpoch()</code> , <code>lossManager.applyLoss()</code> , <code>oracle.performUpkeep()</code>) must strictly increase <code>vault.currentEpochId</code>	Report
p7b_nonAdvancingEpoch	Verified	Methods that are not epoch-advancing (as defined in the previous rule) must leave <code>vault.currentEpochId</code> unchanged	Report Report for setters methods
p7c_onlyYieldOrLossAdvancesEpochLeq	Verified	<code>oracle.submitNav()</code> and <code>umbrella.performUpkeep()</code> advance the epoch monotonically ($\geq$), not strictly, because these methods may or may not trigger an advance depending on pending state	Report Report for setters methods

Tranche

Assumptions and simplifications:

The following assumptions are shared across all properties and rules of the Tranche module.

- no self calls i.e. `Tranche` does not call itself
- total supply of `MZUSD` is non-zero
- `Tranche.asset()` is `MZUSD`

Module Properties

The following properties are verified for the Tranche using property-specific assumptions:

P-24. Exchange rate of Tranche shares only increases except for loss events

- To make verification faster, two helper functions were introduced that calculated the numerator and the denominator of the calculation used in `Tranche.pricePerShare`. `pricePerShareEqualToSummary` proves that `pricePerShare` always corresponds to the division of the numerator and denominator.
- direct calls to `burnMZUSD` from the `LossManager` were filtered since this is only called as part of a loss event (for which a decreasing price is expected)

- direct calls to `notifyYield` from the `IssuanceVault` were filtered out since, as part of `IssuanceVault.distributeYield` the function `MZUSD.mint` is called. Instead we added a harness function called `transferMzusdAndNotifyYieldCVL` which simulates this.

P-25. `userSharesByEpoch` tracked correctly

- for all, after constructor is called, it is assumed for all `epochIds` that:
 - `getFinalizedEpochTotalShares(epochId) == 0`
 - for all `users`, the sum of all `userSharesByEpoch[user][epochId]` is zero

P-60. Integrity of `Tranche.finalizeEpoch`

- Assumes that direct withdraw mode is disabled

P-22. Integrity of Tranche total assets

Status: Verified

Rule Name	Status	Description	Link to rule report
totalAssetsFormula	Verified	<i><code>totalAssets()</code> always returns a value that is equal to MZUSD balance of the contract minus the unaccrued yield minus the reserved assets (finalized but unclaimed umbrella assets)</i>	Report

P-23. Integrity of Tranche `requestWithdrawal`

Status: Verified

Rule Name	Status	Description	Link to rule report
directModelImmediateRedeem	Verified	<i>When direct mode is enabled then Tranche shares are immediately redeemed</i>	Report
umbrellaModeTransfersToSelf	Verified	<i>When umbrella mode is enabled then Tranche shares are sent to the Tranche contract</i>	Report

P-24. Exchange rate of Tranche shares only increases except for loss events

Status: Verified

Rule Name	Status	Description	Link to rule report
pricePerShareNonDecreasing	Verified	<i>pricePerShare</i> is non-decreasing when we filter out direct calls to <i>burnMZUSD</i> and <i>notifyYield</i> .	Report
pricePerShareEqualToSummary	Verified	Calculating the <i>pricePerShare</i> involves a division. Two helper functions calculate the numerator and denominator of the <i>pricePerShare</i> function. This rule ensures that <i>pricePerShare</i> equals numerator / denominator.	Report

P-25. *userSharesByEpoch* tracked correctly

Status: Verified

Rule Name	Status	Description	Link to rule report
userSharesSumEqualsUmbrella	Verified	For a specific epoch, the sum of all <i>userSharesByEpoch</i> values equals the value returned by <i>umbrellaSharesByEpoch</i>	Report

P-26. Deposit limit cannot be inflated

Status: Violated

Rule Name	Status	Description	Link to rule report
depositLimitNotInflatableBy Minting	Violated Reported issue: M-03	When MZUSD is minted the value returned from <code>maxDeposit()</code> cannot increase	Report

P-27. No double claiming

Status: Verified

Rule Name	Status	Description	Link to rule report
claimZeroesUserShares	Verified	For a given epoch, calling <code>claim()</code> zeroes the user's shares.	Report
secondClaimReverts	Verified	For a given epoch, if a user tries to call <code>claim()</code> twice the second call reverts.	Report

P-59. Tranche ERC4626 compatibility

Status: Verified after fix

Rule Name	Status	Description	Link to rule report
maxRedeemIsAchievable	Violated Reported issue L-02	When a user's shares are equal to total supply minus virtual shares then user should be able to redeem Tranche shares equal to <code>maxRedeem()</code>	Report (original) Report (fixed)
maxRedeemZeroInUmbrella Mode	Violated Reported issue L-02	When umbrella mode is enabled then <code>maxRedeem()</code> should return zero	Report (original) Report (fixed)
maxWithdrawIsAchievable	Violated Reported issue L-02	When a user's shares are equal to total supply minus virtual shares then user should be able to withdraw MZUSD equal to <code>maxWithdraw()</code>	Report (original) Report (fixed)
maxWithdrawZeroInUmbrella Mode	Violated Reported issue L-02	When umbrella mode is enabled then <code>maxWithdraw()</code> should return zero	Report (original) Report (fixed)

P-60. Integrity of `Tranche.finalizeEpoch`

Status: Verified

Rule Name	Status	Description	Link to rule report
finalizeEpochReservedExact Amount	Verified	<i>finalizeEpoch(epochId) causes reservedAssets() to increase by an amount equal to the share value (in MZUSD) of umbrellaSharesByEpoch(epochId) before finalization.</i>	Report
finalizeEpochClearsUmbrella AndLocksState	Verified	<i>umbrellaSharesByEpoch(epochId) is zero, and finalizedEpochs(epochId) is non-zero after finalization</i>	Report
onlyRequestWithdrawalCanI ncreaseUmbrellaShares	Verified	<i>Only requestWithdrawal can affect the storage tracking umbrella shares</i>	Report
requestedEpochCannotBeFi nalizedBeforeDelayAndCanA fter	Verified	<i>finalizeEpoch will always revert until vault.currentEpoch() returns an epochId strictly greater than the epoch we wish to finalize.</i>	Report

P-61. Unaccrued yield decreases

Status: Verified

Rule Name	Status	Description	Link to rule report
unaccruedYieldNonIncreasi ngOverTime	Verified	<i>Unaccrued yield never increases during an epoch.</i>	Report

P-66. Anyone can call **deposit**, **withdraw** and **claim**

Status: Verified

Rule Name	Status	Description	Link to rule report
anyoneCanCallDepositWithdrawOrClaim	Verified	<i>After the prover finds a set of valid values that make the function succeed, the sender is changed to an arbitrary user with the same sender-dependent state as the successful caller. The rule then shows this call from an arbitrary sender succeeds, proving that anyone can call the function.</i>	Report

IssuanceVault

P-18. Sum of Tranche BPS values is equal to 10000 BPS

Status: Verified

Rule Name	Status	Description	Link to rule report
rewardAndRiskBPSSumEqualsBPS	Verified	<i>baseRewardBPS plus sum of riskPremiumBps for all tranches equals BPS == 10000 Verified for a single tranche, i.e. tranches.length == 1</i>	Report

P-19. `distributeYield()` can be called at most once per epoch

Status: Verified

Rule Name	Status	Description	Link to rule report
iv_p17_distributeYieldOncePerEpoch	Verified	A second <code>distributeYield()</code> call within the same epoch window (before <code>epochStart + epochDuration</code> has elapsed) reverts.	Report

P-20. Integrity of `rebalance()`

Status: Verified

Rule Name	Status	Description	Link to rule report
integrityOfrebalance	Verified	<code>rebalance()</code> conserves total USDC liquidity (vault + reservoir) across any call, and after the call the vault USDC balance lies within the configured buffer bounds (low, high). No USDC is created or destroyed.	Report

P-21. Valid state conditions and helper invariants

Status: Verified after fix

High-level description

The following rules and invariants define valid states of the system. There are also helper invariants. All of them are used as assumptions for other properties to make the verification more efficient and precise.

Rule Name	Status	Description	Link to rule report
zeroNotRegistered	Verified	<i>address(0) is never set in <code>isTranche</code> mapping</i>	Report
allTranchesMapped	Verified	<i>Every tranche stored in <code>tranches[]</code> has its <code>isTranche[addr]</code> flag set to true</i>	Report
noZeroTranche	Verified	<i>No element of <code>tranches[]</code> is <code>address(0)</code></i>	Report
uniqueTranches	Verified	<i>No tranche address appears more than once in <code>tranches[]</code></i>	Report
feeSumBounded	Verified	<i><code>performanceFeeBps + protocolFeeBps</code> <= <code>MAX_TOTAL_FEE_BPS (5000)</code></i>	Report
feeReceiversNonZero	Verified	<i>Once both <code>performanceFeeReceiver</code> and <code>protocolFeeReceiver</code> are non-zero, they remain non-zero under any call</i>	Report
pairedImpliesinTranches	Verified	<i>Any address <code>a</code> with a non-zero <code>tranchePairedTo[a]</code> appears in <code>tranches[]</code> at a valid witness index</i>	Report
pairedToSymmetric	Verified	<i><code>tranchePairedTo[A] == B</code> implies <code>tranchePairedTo[B] == A</code></i>	Report

pairedRiskPremiumZero	Verified	For every contiguous pair (<code>tranches[i]</code> , <code>tranches[i+1]</code>), the second tranche has <code>riskPremiumBps == 0</code>	Report
epochDurationBounds	Verified	<code>epochDuration</code> is either 0 (pre-initialization) or within <code>[MIN_EPOCH_DURATION=3600 sec, MAX_EPOCH_DURATION=2592000 sec (30 days)]</code>	Report
pendingEpochDurationBounds	Verified	<code>pendingEpochDuration</code> is either 0 (no pending update) or within <code>[MIN_EPOCH_DURATION, MAX_EPOCH_DURATION]</code>	Report
epochIdMonotone	Verified	Once <code>currentEpochId >= 1</code> , it remains <code>>= 1</code> under any call	Report
hysteresisBpsBounded	Verified	<code>hysteresisBps <= BPS (10000)</code> at all times	Report
idleBufferPositive	Verified	Once <code>idleBufferUSDC > 0</code> , it remains <code>> 0</code>	Report
isTrancheBijectiveWithArray	Verified	If <code>isTranche[a]</code> is true then <code>a</code> appears in <code>tranches[]</code> at a valid witness index	Report
pairAdjacent	Verified	For every paired tranche at index <code>i</code> , its partner resides at index <code>i+1</code> or <code>i-1</code>	Report
maxTranchesNotExceeded	Verified	<code>tranches.length <= MAX_TRANCHES (50)</code> at all times	Report
nonZeroFeeBpsRequiresReceiver	Verified after fix Reported issue: I-Q4	<code>performanceFeeBps > 0</code> implies <code>performanceFeeReceiver != 0</code> , and similarly for <code>protocolFeeBps</code>	Report (original) Report (fixed)
setTrancheConfigPreservesLength	Verified	<code>setTrancheConfig()</code> reorders or reconfigures existing elements but never adds or removes	Report

LossManager

P-32. Integrity of `applyLoss()`

Status: Verified after fix

High-level description + impact

`applyLoss()` burns an exact amount of MZUSD and makes the system solvent again. The loss that was not covered by tranches is correctly reflected by `totalUncoveredLossMZ`

Rule Name	Status	Description	Link to rule report
integrityOfApplyLoss	Verified after fix Reported issue: L-10	<code>applyLoss()</code> always moves the system from an insolvent state to the solvent state. I.e., we assume that total supply of MZUSD is greater than the amount of USDC and we expect that enough MZUSD is going to be burned by <code>applyLoss()</code> to make MZUSD backed by USDC again. We used the following assumptions: - <code>oracle.maxAnswerStaleness != 0;</code> - <code>lossManager.totalUncoveredLossMZ == 0;</code> - <code>navOracle.pendingNavOutsideChange < 0 => -navOracle.pendingNavOutsideChange <= navOracle.NAVOutside;</code>	Report (original) Report (fixed)
integrityOfApplyLoss with bound of 1	Verified	<code>applyLoss()</code> always moves the system from an insolvent state to the solvent state w.r.t. one wei. I.e., we assume that total supply of MZUSD is greater than the amount of USDC and we expect that enough MZUSD is going to be burned by <code>applyLoss()</code> to make MZUSD backed by available USDC + 1. We used the following assumptions: - <code>oracle.maxAnswerStaleness != 0;</code> - <code>lossManager.totalUncoveredLossMZ == 0;</code> - <code>navOracle.pendingNavOutsideChange < 0 => -navOracle.pendingNavOutsideChange <= navOracle.NAVOutside;</code>	Report

P-36. No partial loss absorption

Status: Verified

Rule Name	Status	Description	Link to rule report
p36_applyLossRevertsBeyondTotalSupply	Verified	<i>If <code>usdcLoss > mzusd.totalSupply / SCALE</code>, <code>applyLoss()</code> reverts rather than applying the loss to the current supply.</i>	Report

MZUSD

Assumptions and simplifications:

- Reasonable sender assumptions: sender is not `address(0)`, sender is not `ReservoirUSDC`
- The block timestamp is assumed to be less than maximum value of uint40 otherwise unnecessary overflows would occur
- We exclude calls such as `setVault` and `upgradeToAndCall` for the reasons explained earlier.

Module Properties

The following properties are verified for the MZUSD using property-specific assumptions:

P-10 No token loss

- a user transferring `MZUSD` tokens to another address (`MZUSD.[transfer/transferFrom]`)
- a `Tranche` (not a user) instance calling `mint/burn` (which locks up `MZUSD` in return for `Tranche` shares)
 - `withdrawalFeeBps < BPS`
- `MZUSD` calling `IssuanceVault.withdraw`. This is an external call from `MZUSD.requestWithdrawal` and only happens after a corresponding burn of `MZUSD`.
 - restricted operations not called by users
 - `mint` (called by `IssuanceVault`)
 - `burn` (called by `Tranche`).
- Indirect calls to these functions are checked in by adding `IssuanceVault` and `Tranche` contract methods to be checked. We must filter out calls to `deposit` and `mint` because these calls legitimately decrease a user's `MZUSD` balance, without a consequent increase in their `USDC` balance. They increase the user's `Tranche` ERC4626 shares instead.

P-65. Integrity of *mint* and *redeem*

Status: Verified

Rule Name	Status	Description	Link to rule report
depositAuthorizationCheck	Verified	Only users that satisfy <i>allowed(msg.sender)</i> can call <i>deposit</i>	Report
requestWithdrawalAuthorizationCheck	Verified	Only users that satisfy <i>allowed(msg.sender)</i> can call <i>requestWithdrawal</i>	Report
mintAuthorizationCheck	Verified	Only the <i>IssuanceVault</i> can call <i>mint</i>	Report
burnAuthorizationCheck	Verified	Only <i>restricted</i> users can call <i>burn</i>	Report
p6a_onlyDistributeYieldIncreasesSupply	Verified	any method that increases <i>MZUSD totalSupply</i> must be one of <i>vault.distributeYield()</i> , <i>mzUSD.mint()</i> , <i>mzUSD.deposit()</i> , <i>oracle.submitNav()</i> , or <i>oracle.performUpkeep()</i>	Report
p6ai_onlyDistributeYieldIncreasesSupplySetters	Verified	any method that increases <i>MZUSD totalSupply</i> must be one of <i>distributeYield</i> , <i>mzUSD.mint</i> , <i>mzUSD.deposit</i> , <i>oracle.submitNav()</i> , or <i>oracle().performUpkeep</i> . Using setup for setter methods	Report
p6c_mint	Verified	<i>mzUSD.mint()</i> increases <i>balanceOf(to)</i> and <i>totalSupply</i> by exact amount — no hidden fees, rebases, or extra mints. We require the <i>to</i> parameter of <i>mint</i> is not 0.	Report

P-10. No MZUSD token loss

Status: Verified after fix		Rule originally passed, was violated when rounding changed to favour the protocol, the passed again once it reverted on 0 USDC returned for <code>requestWithdrawal</code>	
Rule Name	Status	Description	Link to rule report
noLossOfTokens	Verified after fix	User's <code>MZUSD</code> balance can decrease if-and-only-if their <code>USDC</code> balance increase. Additionally, this particular rule is run on all the methods <code>IssuanceVault</code> and <code>Tranche</code> so as to account for indirect calls to <code>notifyYield</code>	Report

P-11. Rounding in MZUSD always favours the protocol

Status: Verified after fix			
Rule Name	Status	Description	Link to rule report
depositWithdrawDoesNotGain	Verified	A round-trip of deposit followed by <code>requestWithdrawal</code> for the minted <code>MZUSD</code> amount cannot leave the caller with more <code>USDC</code> than they started with, i.e. rounding is in favor of the protocol on full round trips.	Report
depositRoundingAlwaysInFavourOfProtocol	Verified	Splitting a deposit into two calls (<code>deposit(x); deposit(y)</code>) never yields more <code>MZUSD</code> than a single <code>deposit(x+y)</code> of the combined amount — i.e. deposit rounding does not favour the user.	Report
withdrawalRoundingAlwaysInFavourOfProtocol	Verified after fix Reported issue: I-08	Splitting a withdrawal into two calls (<code>requestWithdrawal(x); requestWithdrawal(y)</code>) never yields more <code>USDC</code> than a single <code>requestWithdrawal(x+y)</code> — withdrawal rounding does not favour the user.	Report (original) Report (fixed)

withdrawalRoundingAlwaysInFavourOfProtocolWithinBound	Verified	<i>The gain from splitting the withdrawal into two calls is at most 2 wei of USDC.</i>	Report
--	----------	--	------------------------

P-14. Integrity of *deposit*

Status: Verified

Rule Name	Status	Description	Link to rule report
depositMintsExactScale	Verified	<i>For any <code>deposit(amountUSDC)</code>, the caller's <code>MZUSD</code> balance and <code>totalSupply</code> both increase by exactly <code>amountUSDC * SCALE</code> (where <code>SCALE</code> is sourced from <code>LossManager.SCALE()</code>).</i>	Report

P-15. USDC used in window is never greater than global redemption cap

Status: Verified

Rule Name	Status	Description	Link to rule report
windowUsedIncrementedByEffectiveGross	Verified	<i><code>windowUsedUSDC</code> is never greater than <code>globalRedemptionCapUSDC</code></i>	Report

P-16. **windowStartTimestamp** can only increase

Status: Verified

Rule Name	Status	Description	Link to rule report
blockTimestampGeWindowStartTimestamp	Verified	<i>windowStartTimestamp is always less than or equal to the block timestamp (which means it is monotonically non-decreasing)</i>	Report

P-30. Integrity of **requestWithdrawal**

Status: Verified

Rule Name	Status	Description	Link to rule report
requestWithdrawalBurnsExactAmount	Verified	<i>For any <code>requestWithdrawal(amountMZ)</code>, the caller's MZUSD balance and <code>totalSupply</code> both decrease by exactly <code>floor(amountMZ / SCALE) * SCALE</code> — never more, and rounded down to the nearest multiple of <code>SCALE</code>.</i>	Report

P-38. Integrity of `NavOracleAutomation.performUpkeep`

Status: Verified

Rule Name	Status	Description	Link to rule report
performUpkeepCallsDistributeYieldOrApplyLoss	Verified	When <code>pendingIsYield</code> is true then <code>distributeYield</code> is called, otherwise <code>applyLoss</code> is called.	Report

P-39. `performUpkeep()` or `rejectPendingNav()` clear pendings fields

Status: Verified

Rule Name	Status	Description	Link to rule report
performUpkeepClearsPending	Verified	<code>performUpkeep()</code> clears all pending fields: after a successful call, <code>hasPendingNav()</code> is false and <code>pendingReadyAt</code> , <code>pendingDeltaUSDC</code> , <code>pendingIsYield</code> are all zero.	Report
rejectPendingNavClearsPending	Verified	<code>rejectPendingNav</code> clears all pending fields: after the call, <code>hasPendingNav()</code> is false and <code>pendingReadyAt</code> , <code>pendingDeltaUSDC</code> , <code>pendingIsYield</code> are all zero.	Report

P-40. Integrity of `submitNav()`

Status: Verified

Rule Name	Status	Description	Link to rule report
p40a_submitNavIntegrity_staleness	Verified	- Both oracle answers accepted in <code>submitNav()</code> are no older than <code>maxAnswerStaleness</code> seconds at the time of the call. - <code>submitNav()</code> reverts when the <code>updatedAt</code> timestamps of the NAV oracle answer and the total supply oracle answer differ — both feeds must have been updated in the same on-chain transaction We assume that there are no <code>hasPendingNav</code> and <code>maxAnswerStaleness > 0</code>	Report
p40b_submitNavIntegrity	Verified	- If <code>vetoPeriod == 0</code>, <code>submitNav()</code> applies the NAV immediately in the same transaction (no pending state is left) - <code>pendingReadyAt</code> is fixed at submission time — changing <code>vetoPeriod</code> after <code>submitNav()</code> does not affect the current pending snapshot's readiness timestamp We assume that there are no <code>hasPendingNav</code> and <code>maxAnswerStaleness > 0</code>	Report

P-56. NAV value returned from oracle cannot be used in two epochs

Status: Verified after fix

Rule Name	Status	Description	Link to rule report
maxAnswerStalenessIntegrity	Verified	Ensures that <code>maxAnswerStaleness</code> is strictly less than <code>max(veto, epochDuration)</code> . This rule was added during the re-run.	Report
sameOracleReportNotUsedTwice	Verified after fix Reported issue: M-04	NAV return from a call to <code>AggregatorV3Interface.getRoundData</code> , with a particular <code>updatedAt</code> value, cannot be used in a consecutive epoch because of a <code>maxAnswerStaleness</code> that is too high.	Report (original) Report (fixed)

P-57. Integrity of `checkUpkeep()`

Status: Verified after fix

Rule Name	Status	Description	Link to rule report
p57_checkUpkeepIntegrity	Verified after fix Reported issue: L-07	<code>checkUpkeep()</code> must return false or revert whenever <code>performUpkeep()</code> would revert under the same conditions. We assume the caller is allowed to execute <code>performUpkeep()</code> .	Report

Additionally, we assume that when MZUSD transfers are paused, the protocol is already globally paused. We also assume that whenever rewards are available, there is at least one tranche that is enabled, unpaused, and has active deposits.

PauseManager

P-49. Idempotence of PauseManager

Status: Verified

Rule Name	Status	Description	Link to rule report
pauseGlobalIdempotent	Verified	<code>pauseGlobal()</code> reverts when the category is already globally paused.	Report
pauseContractIdempotent	Verified	<code>pauseContract()</code> reverts when the contract-category pair is already paused.	Report
unpauseGlobalIdempotent	Verified	<code>unpauseGlobal()</code> reverts when the category is not globally paused.	Report
unpauseContractIdempotent	Verified	<code>unpauseContract()</code> reverts when the contract-category pair is not paused.	Report

PolJuniorTranche

Assumptions and simplifications:

The following assumptions are shared across all properties and rules of the PolJuniorTranche module.

- `polPayout` is `address(0)` after the constructor

Module Properties

The following properties are verified for the PolJuniorTranche using property-specific assumptions:

P-47. Integrity of notifyYield

- We assume that `polPayout` is not the PolJuniorTranche contract itself

P-47. Integrity of notifyYield			
Status: Verified			
Rule Name	Status	Description	Link to rule report
notifyYieldTransfersExactAmount	Verified	Exactly the amount passed to <code>notifyYield()</code> is transferred into the junior tranche	Report

P-69. Payout address must not be PolJuniorTranche contract			
Status: Verified after fix			
Rule Name	Status	Description	Link to rule report
polPayoutNotPolTranche	Verified after fix Reported issue: I-06	Ensures that the <code>polPayout</code> address can never be set to the <code>PolJuniorTranche</code> itself	Report (original) Report (fixed)

ReservoirUSDC

Assumptions and simplifications:

The following assumptions are shared across all properties and rules of the ReservoirUSDC module.

- `_isVault(e, vault)` holds (linkage to the IssuanceVault is consistent).
- `e.msg.sender != reservoir` (the reservoir does not call itself).

P-67. Integrity of ReservoirUSDC accounting

Status: Violated

Rule Name	Status	Description	Link to rule report
depositedByMonotone	Verified	<i>depositedBy</i> never decreases	Report
depositedByIncrementsIsExact	Verified	<i>depositedBy</i> increases by the size of the <i>amount</i> parameter and that the balance of the contract also increases by this amount	Report
depositedByIncreasesOnly OnDeposit	Verified	If the vault is the caller, <i>depositedBy</i> increases on a call to deposit by the amount equal to the increase in the contract's balance. If the vault is not the caller, <i>depositedBy</i> doesn't change.	Report
withdrawnToMonotone	Verified	<i>withdrawnTo</i> never decreases	Report
withdrawnToIncrementsIsExact	Violated Reported issue: I-05	<i>withdrawnTo</i> increases by the size of the <i>amount</i> parameter and that the balance of the contract also increases by this amount. If withdrawing to the vault <i>withdrawnTo</i> doesn't change.	Report
withdrawnToIncreasesOnly OnWithdraw	Violated Reported issue: I-05	If not withdrawing to the vault, <i>withdrawnTo</i> increases on a call to <i>withdraw</i> by the amount equal to the decrease in the contract's balance. If withdrawing to the vault <i>withdrawnTo</i> doesn't change.	Report

P-44. Integrity of ReservoirUSDC accounting

Status: Verified

Rule Name	Status	Description	Link to rule report
depositIncreasesReservoirBalance	Verified	<i>The balance of the reservoir goes up the amount deposited.</i>	Report
withdrawTransfersExactAmount	Verified	<i>The user's balance increases by the amount withdrawn.</i>	Report
withdrawReducesReservoirBalance	Verified	<i>The reservoir's balance decreases by the amount withdrawn.</i>	Report
usdcImmutable	Verified	<i>The USDC address can only be set by the constructor.</i>	Report
setVaultRejectsZeroAddress	Verified	<i><code>setVault</code> rejects <code>address(0)</code>.</i>	Report
setVaultIsOneTime	Verified	<i>The <code>vault</code> can only be set once and reverts if tried again.</i>	Report

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.